

Curso: Arquitectura de Software
Laboratorio 3.
Universidad de Antioquia

Guía de Laboratorio: Prueba de Servicios REST con HATEOAS, Docker, MySQL y Kubernetes
Profesor: Diego José Luis Botía V

1. Objetivos

La implementación de esta guía de laboratorio tiene varios objetivos fundamentales que abarcan desde la comprensión teórica hasta la ejecución práctica de aplicaciones en un entorno moderno de desarrollo y despliegue. Los objetivos son los siguientes:

1. **Desarrollar una Aplicación RESTful:** Construir una aplicación en Spring Boot que implemente servicios REST utilizando el principio HATEOAS (Hypermedia as the Engine of Application State).
2. **Contenerización:** Aprender a contenerizar la aplicación y la base de datos utilizando Docker para facilitar la portabilidad y el aislamiento del entorno de ejecución.
3. **Integración con MySQL:** Configurar y utilizar una base de datos MySQL para almacenar datos de manera persistente, garantizando que la aplicación REST pueda interactuar con la base de datos de forma eficiente.
4. **Despliegue en Kubernetes:** Implementar la aplicación y la base de datos en un clúster de Kubernetes, permitiendo la orquestación y gestión automatizada de los contenedores.
5. **Pruebas de Funcionalidad:** Realizar pruebas de la aplicación para asegurar que los servicios REST funcionan correctamente y cumplen con los criterios definidos, incluidos los endpoints HATEOAS.

2. Marco Teórico

2.1 HATEOAS

HATEOAS es un componente clave de la arquitectura REST, que permite a los clientes interactuar con una aplicación a través de hipervínculos que son proporcionados dinámicamente. Esto significa que, además de las respuestas de datos, la aplicación también incluye enlaces a otras acciones posibles, facilitando la navegación y la interacción del cliente con el servicio. Por ejemplo, al obtener información sobre un recurso, el cliente puede recibir enlaces para crear, actualizar o eliminar ese recurso.

2.2 Docker

Docker es una plataforma que permite a los desarrolladores empaquetar aplicaciones y sus dependencias en contenedores ligeros y portables. Los contenedores son entornos de ejecución aislados que garantizan que una aplicación se ejecute de la misma manera sin importar dónde se despliegue.

2.3 Kubernetes

Kubernetes es un sistema de orquestación para la gestión automatizada de aplicaciones en contenedores. Permite el despliegue, escalado y operación de aplicaciones contenerizadas en un clúster de servidores. Kubernetes gestiona automáticamente la carga, la disponibilidad y la recuperación ante fallos, lo que simplifica la administración de aplicaciones complejas. A continuación, se describen algunos elementos básicos de Kubernetes: `kubectl cluster-info`

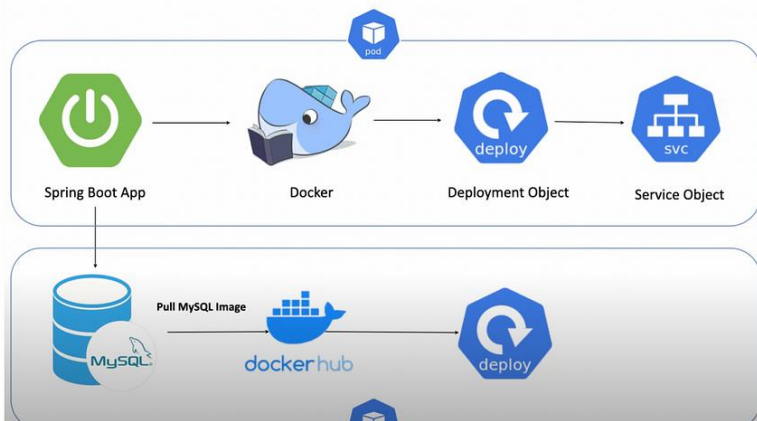
- **Clúster Kubernetes:** Es un conjunto de máquinas (nodos) que trabajan juntas para administrar y ejecutar aplicaciones en contenedores de forma coordinada.
- **Namespaces:** Espacios lógicos que dividen un clúster Kubernetes en dominios virtuales separados, lo que permite aislar y organizar aplicaciones y recursos.
- **Nodos:** Las máquinas físicas o virtuales en un clúster Kubernetes donde se ejecutan los contenedores y se gestionan los recursos.
- **Pods:** La unidad más pequeña desplegable en Kubernetes, que puede contener uno o más contenedores. Los pods comparten recursos y se ejecutan en el mismo nodo.
- **Contenedores:** Paquetes ligeros y portátiles que contienen aplicaciones y todas sus dependencias, facilitando la implementación y gestión de aplicaciones.

2.4 MySQL

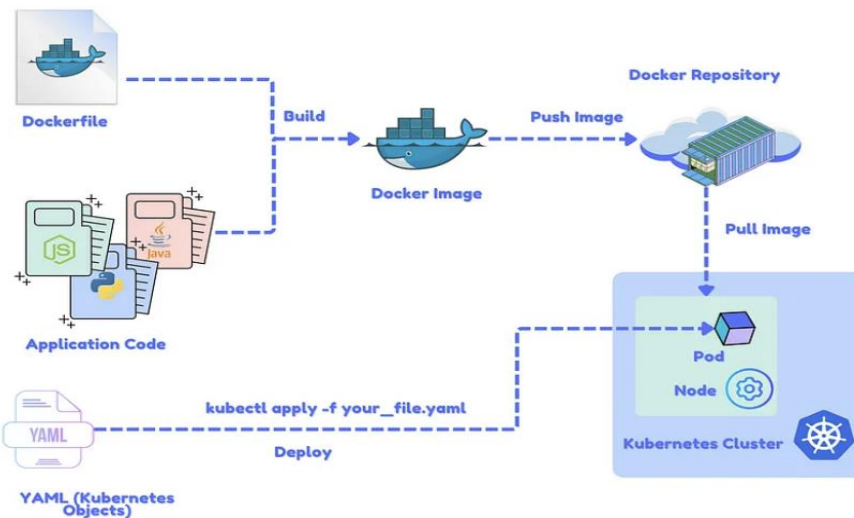
MySQL es un sistema de gestión de bases de datos relacional (RDBMS) que utiliza el lenguaje SQL para gestionar y manipular datos. Es ampliamente utilizado por su facilidad de uso, escalabilidad y robustez. En esta guía, utilizaremos MySQL como nuestra base de datos principal para almacenar la información relacionada con los vuelos.

3. Diagrama de Arquitectura

A continuación, se presenta el diagrama de arquitectura de la solución que se implementará. Este diagrama muestra cómo interactúan los diferentes contenedores, la base de datos MySQL y los servicios de Kubernetes.



Tomado de : <https://blog.devops.dev/deploying-spring-boot-application-on-k8s-1a558d4f965a>



Tomado de : <https://levelup.gitconnected.com/from-localhost-to-the-cloud-deploying-spring-boot-mysql-app-on-kubernetes-with-docker-desktop-a-8c51f9cd23fa>

HERRAMIENTAS

- Cuenta en Github
- Git
- SpringBoot Libraries
- IntelliJ 2024.1 o superior
- JDK 11
- Minikube for Windows
- Kubectl (Kubernetes)
- Docker (<https://www.docker.com/get-started>)
- MySQL
- Swagger u Open API
- Postman (<https://www.postman.com/>)

CREANDO LA APLICACIÓN

Vamos a crear una aplicación web con Spring Boot que contenga las siguientes características:

- Debe permitir realizar la creación de vuelos en una base de datos MySQL.
- Deberá proveer de un listado de todos los vuelos.
- Deberá proveer la información de un vuelo según un ID específico.
- Debe retornar los mejores vuelos según un rating.
- Se declara un Controller (FlightController) que posee todos los endpoints necesarios.

Curso: Arquitectura de Software

Laboratorio 3.

Universidad de Antioquia

Spring Boot provee un asistente en donde uno puede seleccionar qué módulos va a utilizar en una aplicación determinada y tras ello tendremos disponible el “esqueleto” de dicha aplicación para descargar. Se trata de Spring Initializr (<https://start.spring.io/>). Una de las ventajas de Spring Boot es que proporciona paquetes iniciales que simplifica su configuración de Maven. Los iniciadores Spring Boot son en realidad un conjunto de dependencias que puede incluir en su proyecto. Para nuestro ejemplo vamos a indicar el grupo, artefacto y dependencias (Rest, HAL, HATEOAS, Lombok, Web, Swagger, Actuador, MySQL Driver) que utilizaremos como sigue:

Project

☐ Gradle - Groovy ☐ Gradle - Kotlin ☒ **Java** ☐ Kotlin ☐ Groovy

☒ **Maven**

Spring Boot

☐ 3.5.0 (SNAPSHOT) ☐ 3.5.0 (RC1) ☐ 3.4.6 (SNAPSHOT) ☐ 3.4.5 ☐ 3.3.12 (SNAPSHOT) ☒ **3.3.11**

Project Metadata

Group

com.udea

Artifact

kbt

Name

kbt

Description

Demo project for Spring Boot

Package name

com.udea.kbt

Packaging

☒ **Jar** ☐ War

Java

☐ 24 ☐ 21 ☒ **17**

Dependencies

ADD DEPENDENCIES... CTRL + B

Lombok **DEVELOPER TOOLS**

Java annotation library which helps to reduce boilerplate code.

Spring Web **WEB**

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Rest Repositories **WEB**

Exposing Spring Data repositories over REST via Spring Data REST.

Rest Repositories HAL Explorer **WEB**

Browsing Spring Data REST repositories in your browser.

Spring HATEOAS **WEB**

Eases the creation of RESTful APIs that follow the HATEOAS principle when working with Spring / Spring MVC.

Spring Data JPA **SQL**

Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

MySQL Driver **SQL**

MySQL JDBC driver.

Validation **IO**

Bean Validation with Hibernate validator.

Spring Boot Actuator **OPS**

Supports built in (or custom) endpoints that let you monitor and manage your application - such as application health, metrics, sessions, etc.

Spring Boot Dev Tools **DEVELOPER TOOLS**

Provides fast application restarts, LiveReload, and configurations for enhanced development experience.

Nota: Puede usar también Java 11 o 17 según sea su caso.

DEPENDENCIAS

Lombok: Es una librería java que contiene anotaciones que generan automáticamente métodos de ayuda después de la compilación como por ejemplo (setters, getters, constructores).

Spring Web: Proporciona características de integración comunes específicas de la web, para construir aplicaciones web, incluyendo RESTful, utilizando Spring MVC.

Spring Data JPA: Proporciona soporte de repositorio para Java Persistence API (JPA). Facilitando el desarrollo de aplicaciones que necesitan acceder a fuentes de datos JPA.

Spring Boot DevTools: Proporciona reinicio rápido de las aplicaciones.

Validator: Es el estándar para implementar la lógica de validación en el ecosistema Java, por ejemplo, lo utilizamos en la clase validando los campos con sus respectivas restricciones. Por otro lado, está bien integrado con Spring y Spring Boot.

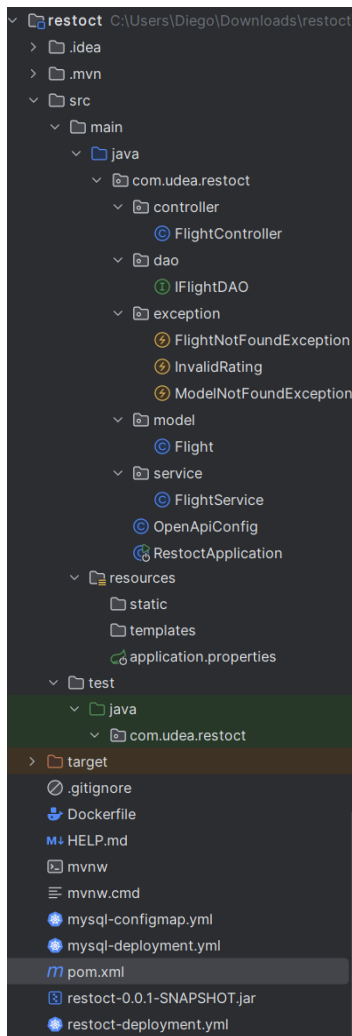
MySQL Driver: Driver que proporciona acceso a MySQL.

Spring-boot-starter-parent: Todos los proyectos de spring boot construidos en maven utilizan spring boot starter parent en su pom.xml. Estas son algunas acciones que realiza :

- Configurar la versión y las propiedades de Java.
- Gestionar las dependencias y sus versiones.

Una vez generado este proyecto vamos a obtener un archivo .zip que podemos descargar y descomprimir en cualquier ubicación. Al abrir la carpeta descomprimida con un IDE de desarrollo podemos ver la estructura de directorios creada por Spring Boot. Si utiliza Netbeans puede ir al menú **File** y seleccionar **Import Zip File**. Como se puede observar la siguiente figura, la estructura de la aplicación estará dividida por paquete.

Curso: Arquitectura de Software
Laboratorio 3.
Universidad de Antioquia



CONTENIDO DE POM.XML

Hemos seleccionado Maven como manejador de dependencias al generar nuestro proyecto en Spring Initializr, no obstante, todas las dependencias elegidas de Spring Boot podremos verlas en el archivo pom.xml dentro del nodo <dependencies> como nuestro a continuación:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>2.7.17</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.udea</groupId>
    <artifactId>restoct</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>restoct</name>
    <description>Demo project for Spring Boot</description>
    <properties>
```

Curso: Arquitectura de Software
Laboratorio 3.
Universidad de Antioquia

```
<java.version>11</java.version>
</properties>
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-rest</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-hateoas</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.data</groupId>
    <artifactId>spring-data-rest-hal-explorer</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-ui</artifactId>
    <version>1.7.0</version> <!-- Usa la última versión disponible -->
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-configuration-processor</artifactId>
    <optional>true</optional>
  </dependency>
  <dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>5.1.34</version>

  </dependency>
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <version>1.18.34</version> <!-- Usa la última versión -->
    <scope>provided</scope>

  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <configuration>
        <excludes>
```

Curso: Arquitectura de Software
Laboratorio 3.
Universidad de Antioquia

```
                <exclude>
                    <groupId>org.projectlombok</groupId>
                    <artifactId>lombok</artifactId>
                </exclude>
            </excludes>
        </configuration>
    </plugin>
</plugins>
</build>
</project>
```

ALGUNOS ARCHIVOS IMPORTANTES

En nuestra aplicación generada por defecto podemos encontrar algunos archivos que serán de importancia en un futuro:

- `src/main/java/com/udea/FlightApplication.java`: Contiene el script de arranque (main) de la aplicación spring boot.
- `src/main/resources/application.properties`: En este archivo podremos configurar cualquier tipo de parámetros (conexión con la base de datos, variables utilizadas en el código, etc).

CONFIGURANDO EL ACCESO A DATOS

Con respecto a la configuración del acceso a datos tenemos que tener en cuenta los siguientes componentes a crear:

- Configuración de los parámetros de conexión con la base de datos:
Particularmente, en el archivo **application.properties** debemos indicar dichos parámetros para conectarnos a MySQL:

```
spring.application.name=restoct

server.port=8089
#spring.datasource.url=jdbc:mysql://localhost:3306/restoct
spring.datasource.url=jdbc:mysql://mysql:3306/restoct
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL5Dialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

management.endpoints.web.exposure.include=*
# HEALTH ENDPOINT
management.endpoint.health.show-details=always
```

- Creación de clase en el dominio: Necesitamos crear una entidad bajo el nombre de **"Flight"** la cual será mapeada con la respectiva colección en la base de datos. Esta clase de modelo representa un Vuelo. Esta clase contiene un id de un vuelo que se auto genera, con sus respectivos número de vuelo, nombre, ciudad origen y destino, numero de plan de vuelo, rating entre otros :

```
package com.udea.kbt.model;

//import jakarta.persistence.*;

import lombok.*;
import org.hibernate.proxy.HibernateProxy;

import javax.persistence.*;
import javax.validation.constraints.Max;
import javax.validation.constraints.Min;
import java.io.Serializable;
import java.util.Objects;

@Data
@RequiredArgsConstructor
@NoArgsConstructor
@AllArgsConstructor
@Entity
```

Curso: Arquitectura de Software
Laboratorio 3.
Universidad de Antioquia

```
public class Flight implements Serializable {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    @Column(name="idflight")
    private Long idFlight;

    @Column(name="nombreavion", nullable=false, length=80)
    private @NonNull String nombreAvion;

    // @ApiModelProperty(notes = "Numero Vuelo")
    @Column(name="numerovuelo", nullable=false, length=80)
    private @NonNull String numeroVuelo;

    @Column(name="origen", nullable=false, length=80)
    private @NonNull String origen;

    @Column(name="destino", nullable=false, length=80)
    private @NonNull String destino;

    @Column(name="capacidad", nullable=false, length=80)
    private @NonNull int capacidad;

    @Column(name="rating", nullable=false, length=80)
    @Min(value=1, message="id should be more or than equal 1")
    @Max(value=5, message="id should be less or than equal 5")
    private @NonNull int rating;

    @Column(name="planvuelo", nullable=false, length=80)
    private @NonNull long planvuelo;
    private Boolean cumplido;

    public Long getIdFlight() {
        return idFlight;
    }

    public void setIdFlight(Long idFlight) {
        this.idFlight = idFlight;
    }

    public @NonNull String getNombreAvion() {
        return nombreAvion;
    }

    public void setNombreAvion(@NonNull String nombreAvion) {
        this.nombreAvion = nombreAvion;
    }

    public @NonNull String getNumeroVuelo() {
        return numeroVuelo;
    }

    public void setNumeroVuelo(@NonNull String numeroVuelo) {
        this.numeroVuelo = numeroVuelo;
    }

    public @NonNull String getOrigen() {
        return origen;
    }

    public void setOrigen(@NonNull String origen) {
        this.origen = origen;
    }

    public @NonNull String getDestino() {
        return destino;
    }

    public void setDestino(@NonNull String destino) {
        this.destino = destino;
    }
}
```

Curso: Arquitectura de Software
Laboratorio 3.
Universidad de Antioquia

```
    }

    @NonNull
    public int getCapacidad() {
        return capacidad;
    }

    public void setCapacidad(@NonNull int capacidad) {
        this.capacidad = capacidad;
    }

    @Min(value = 1, message = "id should be more or than equal 1")
    @Max(value = 5, message = "id should be less or than equal 5")
    @NonNull
    public int getRating() {
        return rating;
    }

    public void setRating(@Min(value = 1, message = "id should be more or than equal 1") @Max(value = 5,
message = "id should be less or than equal 5") @NonNull int rating) {
        this.rating = rating;
    }

    @NonNull
    public long getPlanvuelo() {
        return planvuelo;
    }

    public void setPlanvuelo(@NonNull long planvuelo) {
        this.planvuelo = planvuelo;
    }

    public Boolean getCumplido() {
        return cumplido;
    }

    public void setCumplido(Boolean cumplido) {
        this.cumplido = cumplido;
    }
}
```

Note que usaremos la librería Lombok que nos permitirá simplificar nuestra entidad con el uso de anotaciones simples.

- Creación de la Interface Repository: Necesitamos crear una interfaz **IFlightDAO** que extienda de **CrudRepository** para realizar las operaciones CRUD contra el objeto **Flight**:

```
package com.udea.restoct.dao;

import com.udea.restoct.model.Flight;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.CrudRepository;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface IFlightDAO extends CrudRepository<Flight, Long> {
    @Query("from Flight f where f.rating>=4 AND f.cumplido=true")
    public List<Flight> viewBestFlight();
}
```

La clase de **FlightService** facilita la comunicación del repositorio con las operaciones de guardar, actualizar, eliminar, listar por **IdFlight**, listar todos los vuelos y listar los mejores vuelos calificados

```
package com.udea.restoct.service;

import com.udea.restoct.dao.IFlightDAO;
import com.udea.restoct.exception.FlightNotFoundException;
```


Curso: Arquitectura de Software
Laboratorio 3.
Universidad de Antioquia

```
import com.udea.restoct.model.Flight;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

@Service
public class FlightService {

    @Autowired
    private IFlightDAO dao;

    public Flight save(Flight t) {
        return dao.save(t);
    }

    public String delete(long id) {
        dao.deleteById(id);
        return "Flight removed";
    }

    public Iterable<Flight> list() {
        return dao.findAll();
    }

    public Optional<Flight> listId(long id) {
        return dao.findById(id);
    }

    public Flight update(Flight t) {
        Flight existingFlight = dao.findById(t.getIdFlight()).orElse(null);
        existingFlight.setNombreAvion(t.getNombreAvion());
        existingFlight.setNumeroVuelo(t.getNumeroVuelo());
        existingFlight.setOrigen(t.getOrigen());
        existingFlight.setDestino(t.getDestino());
        existingFlight.setRating(t.getRating());
        existingFlight.setPlanvuelo(t.getPlanvuelo());
        existingFlight.setCapacidad(t.getCapacidad());
        existingFlight.setCumplido(t.getCumplido());
        return dao.save(existingFlight);
    }

    public List<Flight> viewBestFlight() throws FlightNotFoundException {
        List<Flight> flights = dao.viewBestFlight();
        if (flights.size() > 0)
            return flights;
        else
            throw new FlightNotFoundException("No Flight found with rating>=4");
    }
}
```

DECLARACIÓN DEL CONTROLADOR

Los servicios web son aplicaciones que se comunican a través de Internet utilizando el protocolo HTTP. Hay muchos tipos diferentes de arquitecturas de servicios web, pero la idea principal en todos los diseños es la misma, aquí estamos creando un servicio web RESTful a partir de lo que es un diseño, con el objetivo de ver la información de un vuelo. Crear un endpoint permite crear un nuevo registro del vuelo en el sistema, también existe la opción de un endpoint, donde devuelve una respuesta JSON para visualizar la información de todos los vuelos disponibles en la aplicación, ver/{id} permite buscar información del vuelo con el id apropiado. Puede agregar nuevos endpoints si lo desea.

Necesitamos ahora crear la clase **FlightController**, la cual tendrá los endpoints necesarios para realizar las operaciones de la base de datos. Este archivo tendrá como dependencia a **FlightService** creado anteriormente:

```
package com.udea.restoct.controller;

import com.udea.restoct.exception.InvalidRating;
import com.udea.restoct.exception.ModelNotFoundException;
```

Curso: Arquitectura de Software
Laboratorio 3.
Universidad de Antioquia

```
import com.udea.restoct.model.Flight;
import com.udea.restoct.service.FlightService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/flight")
@CrossOrigin("")
public class FlightController {
    @Autowired
    FlightService flightService;

    @PostMapping("/save")
    public long save(
        @RequestBody Flight flight) throws InvalidRating {
        if(flight.getRating() > 5)
            throw new InvalidRating("Id should be less than or equal 5");
        flightService.save(flight);
        return flight.getIdFlight();
    }

    @GetMapping("/listAll")
    public Iterable<Flight> listAllFlights(){return flightService.list();}

    @GetMapping("/list/{id}")
    public Flight listFlightById( @PathVariable("id") int id){
        Optional<Flight> flight= flightService.listId(id);
        if(flight.isPresent()){
            return flight.get();
        }

        throw new ModelNotFoundException("ID de Flight invalido");
    }

    @GetMapping("/topFlights")
    public ResponseEntity<List<Flight>> viewBestFlights(){
        List<Flight> list=flightService.viewBestFlight();
        return new ResponseEntity<List<Flight>>(list, HttpStatus.ACCEPTED);
    }

    @PutMapping
    public Flight updateFlight(@RequestBody Flight flight){
        return flightService.update(flight);
    }

    @DeleteMapping("/{id}")
    public String deleteFlight(@PathVariable long id){
        return flightService.delete(id);
    }
}
```

Ahora necesitamos una clase que gestione las excepciones. **ModelNotFoundException** lanzará una excepción de tiempo de ejecución personalizada si el identificador del conductor no existe en la aplicación.

```
package com.udea.restoct.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(HttpStatus.NOT_FOUND)
public class ModelNotFoundException extends RuntimeException {
    public ModelNotFoundException(String mensaje) {
        super(mensaje);
    }
}
```

Curso: Arquitectura de Software
Laboratorio 3.
Universidad de Antioquia

```
}  
}
```

También podremos crear otros controladores de excepciones como por ejemplo si el valor del rating es incorrecto o si no se encuentran vuelos. Puede crear las siguientes clases :

```
package com.udea.restoct.exception;  
  
import org.springframework.http.HttpStatus;  
import org.springframework.web.bind.annotation.ResponseStatus;  
  
@ResponseStatus(HttpStatus.NOT_FOUND)  
public class FlightNotFoundException extends RuntimeException {  
    public FlightNotFoundException(String mensaje) {  
        super(mensaje);  
    }  
}  
  
package com.udea.restoct.exception;  
  
public class InvalidRating extends RuntimeException {  
  
    public InvalidRating() {  
        super();  
        // TODO Auto-generated constructor stub  
    }  
  
    public InvalidRating(String message) {  
        super(message);  
        // TODO Auto-generated constructor stub  
    }  
}
```

Configuración de Swagger 2 en la aplicación

Springfox proporciona una anotación **@EnableSwagger2** que indica que el soporte de Swagger debe estar activado. Esto debería aplicarse a una configuración Spring Java y debería tener una anotación **@Configuration**.

Vamos a crear un Docket bean en una configuración de Spring Boot para configurar Swagger 2 para la aplicación. Una instancia de Docket Springfox proporciona la configuración primaria de la API con valores predeterminados y métodos prácticos para la configuración.

Vamos a personalizar Swagger proporcionando información sobre nuestra API en la clase Swagger2Config de esta manera.

```
package com.udea.restoct;  
  
import org.springframework.context.annotation.Bean;  
import org.springframework.context.annotation.Configuration;  
  
import io.swagger.v3.oas.models.OpenAPI;  
import io.swagger.v3.oas.models.info.Info;  
  
@Configuration  
public class OpenApiConfig {  
  
    @Bean  
    public OpenAPI customOpenAPI() {  
        return new OpenAPI()  
            .info(new Info()  
                .title("API de Gestión de Vuelos")  
                .version("1.0")  
                .description("Documentación de la API de vuelos para gestionar información de  
vuelos"));  
    }  
}
```

Para ejecutar la UI de Swagger puede usar la siguiente URL <http://localhost:8089/swagger-ui/index.html>

Curso: Arquitectura de Software
Laboratorio 3.
Universidad de Antioquia

Observará la siguiente salida donde podrá incluso probar las APIs expuestas.

Spring Boot REST API 1.0.0

[Base URL: localhost:8089/]
<http://localhost:8089/v2/api-docs>

Flight Management REST API

[ee@gmail.com](#) - Website
Send email to [ee@gmail.com](#)
[Apache 2.0](#)

flight-controller

Operations to Flights

PUT

/flight

updateFlight

DELETE

/flight/{id}

deleteFlight

GET

/flight/list/{id}

Get Flight by ID

GET

/flight/listAll

View a list of available flights

POST

/flight/save

Add Flight

GET

/flight/topFlights

Get Top Flights

Curso: Arquitectura de Software
Laboratorio 3.
Universidad de Antioquia

GET

/flight/listAll

View a list of available flights

Parameters

No parameters

Execute

Responses

Curl

curl -X GET "http://localhost:8089/flight/listAll" -H "accept: /*"

Request URL

http://localhost:8089/flight/listAll

Server response

Code	Details
200	Response body <pre>[{ "idFlight": 1, "nombreAvion": "A320", "numeroVuelo": "AA007", "origen": "Medellin", "destino": "cali", "capacidad": 200, "rating": 4, "planvuelo": 23234, "cumplido": true }]</pre> Response headers <pre>connection: keep-alive content-type: application/json date: Sun14 Apr 2024 01:52:26 GMT keep-alive: timeout=60 transfer-encoding: chunked vary: OriginAccess-Control-Request-MethodAccess-Control-Request-Headers</pre>

Curso: Arquitectura de Software
Laboratorio 3.
Universidad de Antioquia

POST

/flight/save

Add Flight

Parameters

Ca

Name	Description
flight * required	Flight Object Store in DB table
object (body)	Edit Value Model <pre>{ "capacidad": 200, "cumplido": true, "destino": "cali", "nombreAvion": "A320", "numeroVuelo": "AA807", "origen": "Medellin", "planvuelo": 23234, "rating": 4 }</pre> Cancel Parameter content type application/json

Execute

Clear

Responses

Response content type

*/

Curl

```
curl -X POST "http://localhost:8089/flight/save" -H "accept: */*" -H "Content-Type: application/json" -d '{"capacidad": 200, "cumplido": true, "destino": "cali", "nombreAvion": "A320", "numeroVuelo": "AA807", "origen": "Medellin", "planvuelo": 23234, "rating": 4}'
```

ARRANQUE DE APLICACIÓN Y PRUEBA DE FUNCIONAMIENTO

Para arrancar la aplicación es necesario correr el siguiente comando desde la raíz del proyecto:

mvn spring-boot:run

Esto iniciara el servidor Tomcat en el puerto 8089 con acceso a todos los endpoints creados en el controller del punto anterior.

En caso de que, por algún motivo, se desee cambiar el servidor en el cual se levanta el Tomcat, será necesario agregar la entrada "server.port" al application.properties y colocar el puerto que se crea conveniente.

Podra acceder al navegador y observe que se carga por defecto el navegador HAL (HyperText Application Language) donde podrá probar los links que definen los llamados a las interfaces REST de su aplicación.

Curso: Arquitectura de Software
Laboratorio 3.
Universidad de Antioquia

The screenshot shows 'The HAL Browser (for Spring Data REST)' interface. The address bar shows 'http://192.168.99.100:8089/personae'. The 'Explorer' panel on the left shows a 'Custom Request Headers' section, 'Properties' (empty), and 'Links' table:

rel	title	name / index	docs	GET	NON-GET
self				→	!
profile				→	!

Below the links is an 'Embedded Resources' section with a list: 'personae[0]' and 'personae[1]'. The 'Inspector' panel on the right shows 'Response Headers' (200 success, content-type: application/hal+json; charset=UTF-8) and 'Response Body' (JSON data for two persons: Diego Botia and Juan Gomez).

Una vez iniciada la aplicación ya estamos en condiciones de realizar las pruebas de nuestro servicio REST. En nuestro caso utilizando Postman para realizar estas pruebas, pero puede utilizarse cualquier otra herramienta (incluso crear una aplicación frontend que consuma estos servicios, obviamente).

Configuración de Pruebas en Postman

Asegúrese de que el servidor esté en ejecución y de que la URL base coincida con la que está en su entorno (por ejemplo, http://localhost:8080). Reemplace localhost:8080 en las solicitudes si tu servidor usa una URL o puerto diferente.

1. Guardar un Vuelo (POST /flight/save)

- **Método:** POST
- **URL:** http://localhost:8089/flight/save
- **Headers:**
 - Content-Type: application/json
- **Body:** (JSON)

```
{
  "nombreAvion": "Airbus A320",
  "numeroVuelo": "AV123",
  "origen": "Bogotá",
  "destino": "Medellín",
  "rating": 4,
  "planvuelo": "Plan A",
  "capacidad": 180,
  "cumplido": true
}
```

- **Respuesta Esperada:** Código de estado 200 OK con el idFlight del vuelo recién creado.

2. Listar Todos los Vuelos (GET /flight/listAll)

- **Método:** GET
- **URL:** http://localhost:8089/flight/listAll
- **Headers:** No se requiere
- **Respuesta Esperada:** Código de estado 200 OK con un arreglo de objetos Flight.

3. Obtener Vuelo por ID (GET /flight/list/{id})

- **Método:** GET
- **URL:** http://localhost:8089/flight/list/{id}
 - Reemplace {id} con el idFlight del vuelo que desee consultar.
- **Headers:** No se requiere

Curso: Arquitectura de Software
Laboratorio 3.
Universidad de Antioquia

- **Respuesta Esperada:** Código de estado 200 OK con los detalles del vuelo. Si el id no existe, se espera una respuesta 404 con el mensaje de error "ID de Flight invalido".

4. Obtener los Mejores Vuelos (GET /flight/topFlights)

- **Método:** GET
- **URL:** http://localhost:8089/flight/topFlights
- **Headers:** No se requiere
- **Respuesta Esperada:** Código de estado 202 ACCEPTED con una lista de vuelos con una calificación igual o mayor a 4.

5. Actualizar Vuelo (PUT /flight)

- **Método:** PUT
- **URL:** http://localhost:8089/flight
- **Headers:**
 - Content-Type: application/json
- **Body:** (JSON) Asegurese de incluir el campo idFlight para identificar el vuelo que desea actualizar.

json

Copiar código

```
{
  "idFlight": 1,
  "nombreAvion": "Boeing 737",
  "numeroVuelo": "AV456",
  "origen": "Medellín",
  "destino": "Cartagena",
  "rating": 5,
  "planvuelo": "Plan B",
  "capacidad": 150,
  "cumplido": false
}
```

- **Respuesta Esperada:** Código de estado 200 OK con el objeto Flight actualizado.

6. Eliminar Vuelo (DELETE /flight/{id})

- **Método:** DELETE
- **URL:** http://localhost:8089/flight/{id}
 - Reemplace {id} con el idFlight del vuelo que desee eliminar.
- **Headers:** No se requiere
- **Respuesta Esperada:** Código de estado 200 OK con el mensaje "Flight removed".

4.1 ConfigMap para MySQL

El ConfigMap permite almacenar configuraciones de la base de datos MySQL que pueden ser utilizadas por los contenedores. Esto facilita la gestión de parámetros de configuración sin tener que modificar los contenedores.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: mysql-config
data:
  MYSQL_DATABASE: "restoct"
  #MYSQL_USER: "root"
  #MYSQL_PASSWORD: "root"
  MYSQL_ROOT_PASSWORD: "root"
```

□ **apiVersion: v1**

- Define la versión de la API de Kubernetes que se está utilizando para crear este recurso. Aquí se utiliza v1, que es la versión básica y común para objetos como ConfigMap.

□ **kind: ConfigMap**

- Indica que el recurso que se está creando es de tipo ConfigMap. Este recurso permite almacenar datos de configuración clave-valor que pueden ser usados por los contenedores en el clúster.

□ **metadata**

- Aquí se configuran los metadatos del ConfigMap, incluyendo el nombre que lo identifica.
- **name:** Asigna el nombre mysql-config a este ConfigMap, que servirá para referenciarlo en otras partes de la configuración del clúster.

□ **data**

- En esta sección se definen las claves y valores de configuración necesarios para la base de datos MySQL. Estos valores se pasarán como variables de entorno a los contenedores que usen este ConfigMap.

Curso: Arquitectura de Software
Laboratorio 3.
Universidad de Antioquia

- **MYSQL_DATABASE:** Define el nombre de la base de datos MySQL que se debe crear. En este caso, la base de datos se llamará restoct.
- **MYSQL_ROOT_PASSWORD:** Define la contraseña para el usuario root de MySQL. En este ejemplo, la contraseña está configurada como "root".

4.2 Deployment para MySQL

El Deployment para MySQL asegura que haya una instancia de la base de datos ejecutándose. Kubernetes se encargará de crear y gestionar este contenedor. La sección service de MySQL permite acceder a la base de datos desde otros servicios dentro del clúster de Kubernetes. Esto es esencial para que la aplicación REST pueda interactuar con la base de datos.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
spec:
  selector:
    matchLabels:
      app: mysql
  replicas: 1
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:5.7
          envFrom:
            - configMapRef:
                name: mysql-config
          ports:
            - containerPort: 3306
---
apiVersion: v1
kind: Service
metadata:
  name: mysql
spec:
  ports:
    - port: 3306
      targetPort: 3306
  selector:
    app: mysql
  type: ClusterIP
```

1. apiVersion: v1

- Define la versión de la API de Kubernetes para este recurso. v1 es la versión estándar para Service.

2. kind: Service

- Este recurso es un Service, el cual permite la comunicación de la aplicación con otros servicios dentro del clúster.

3. metadata

- Incluye metadatos del Service.
- name: Asigna el nombre mysql al Service.

4. spec

- Define la configuración específica del Service.
- ports:
 - Define el puerto 3306 para el Service, rediriéndolo al mismo puerto 3306 del contenedor, donde MySQL escucha las conexiones.
- selector:
 - Usa el selector app: mysql para enlazar este Service con los Pods que tienen esta etiqueta, lo que garantiza que el tráfico enviado al servicio llegue a los Pods correctos.
- type: ClusterIP:
 - Define el tipo del servicio como ClusterIP, lo que significa que el Service solo será accesible dentro del clúster de Kubernetes, proporcionando una red interna para otros Pods que necesiten conectarse a MySQL.

4.3 Deployment para la Aplicación REST

Curso: Arquitectura de Software
Laboratorio 3.
Universidad de Antioquia

El Deployment para la aplicación REST asegura que se ejecuten múltiples instancias de la aplicación, lo que mejora la disponibilidad y la capacidad de respuesta. El Service de la aplicación REST expone la aplicación a través de un LoadBalancer, permitiendo el acceso desde el exterior del clúster.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kbt-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: kbt
  template:
    metadata:
      labels:
        app: kbt
    spec:
      containers:
        - name: kbt-app
          image: kbt-app:latest
          imagePullPolicy: Never
          ports:
            - containerPort: 8089
      env:
        - name: SPRING_DATASOURCE_URL
          value: "jdbc:mysql://mysql:3306/kbt"
        - name: SPRING_DATASOURCE_USERNAME
          value: root
        - name: SPRING_DATASOURCE_PASSWORD
          valueFrom:
            configMapKeyRef:
              name: mysql-config
              key: MYSQL_ROOT_PASSWORD
---
apiVersion: v1
kind: Service
metadata:
  name: kbt-service
spec:
  ports:
    - port: 8089
      targetPort: 8089
  selector:
    app: kbt
  type: LoadBalancer
```

apiVersion: apps/v1

- Define la versión de la API de Kubernetes, en este caso, la API apps/v1 usada para gestionar Deployments.

2. kind: Deployment

- Declara que este recurso es un Deployment, que se utiliza para administrar y desplegar múltiples réplicas de Pods.

3. metadata

- Incluye metadatos del Deployment.
- **name:** Asigna el nombre kbt-app al Deployment.

4. spec

- Define la configuración específica del Deployment.
- **replicas:**
 - Define que se crearán dos réplicas (Pods) de la aplicación para balancear carga y mejorar disponibilidad.
- **selector:**
 - Usa matchLabels con app: kbt, que selecciona los Pods que tengan esta etiqueta. Esto asegura que el Deployment solo gestione Pods que cumplan este criterio.
- **template:**
 - Define la plantilla que se usará para crear los Pods.
 - **metadata:**
 - Establece la etiqueta app: kbt en los Pods.

Curso: Arquitectura de Software
Laboratorio 3.
Universidad de Antioquia

- **spec:**
 - Describe los contenedores que se ejecutarán en cada Pod.
 - **containers:**
 - Define un único contenedor llamado kbt-app.
 - **image:**
 - Especifica la imagen Docker kbt-app:latest, que contiene el código de la aplicación.
 - **imagePullPolicy:**
 - Usa Never, indicando que Kubernetes no intentará descargar la imagen y usará la versión local.
 - **ports:**
 - Expone el puerto 8089 dentro del contenedor, donde la aplicación estará escuchando.
 - **env:**
 - Define variables de entorno para la configuración de la base de datos:
 - **SPRING_DATASOURCE_URL:** Define la URL de conexión a la base de datos MySQL.
 - **SPRING_DATASOURCE_USERNAME** y **SPRING_DATASOURCE_PASSWORD:** Extraen valores del ConfigMap mysql-config mediante valueFrom y las claves MYSQL_USER y MYSQL_PASSWORD.

Archivo Dockerfile

Ahora creamos el archivo Dockerfile para poder crear la imagen de la aplicación.

```
# Usar la imagen base de OpenJDK
FROM openjdk:11-ea-19-jre-slim

# Establecer el directorio de trabajo
WORKDIR /app

# Copiar el archivo JAR generado por Spring Boot
COPY target/kbt-0.0.1-SNAPSHOT.jar app.jar

# Exponer el puerto de la aplicación
EXPOSE 8089

# Ejecutar la aplicación
ENTRYPOINT ["java", "-jar", "app.jar"]
```

5. Comandos a Ejecutar

Esta sección cubre todos los comandos necesarios para construir y ejecutar la aplicación y la base de datos en el entorno de Kubernetes.

Instalación y configuración de Kubernetes local

Para configurar Minikube y kubectl en Windows 10 u 11 y verificar que ambos funcionen correctamente siga los siguientes pasos:

1. Prerrequisitos

- Asegúrese de tener instalado Windows 10 o superior.
- Instale VirtualBox o Hyper-V como hipervisor, o también puede usar Docker Desktop si ya está instalado, ya que Minikube necesita un hipervisor para crear la máquina virtual.

2. Instalar kubectl

kubectl es la herramienta de línea de comandos para interactuar con Kubernetes.

1. Descargue **kubectl** ejecutando el siguiente comando en Windows PowerShell:

```
curl -LO "https://dl.k8s.io/release/$(curl -s https://dl.k8s.io/release/stable.txt)/bin/windows/amd64/kubectl.exe"
```

2. Agregue **kubectl** al PATH:
 - Mueva kubectl.exe a un directorio de su elección, por ejemplo C:\kubectl.
 - Agregue este directorio a las variables de entorno del sistema (PATH).
3. Verifique la instalación:

kubectl version --client --short

Si la instalación fue exitosa, vera la versión del cliente de kubectl.

Curso: Arquitectura de Software
Laboratorio 3.
Universidad de Antioquia

```
PS C:\Users\Diego\Downloads\restoct\restoct> kubectl version --client --short
Flag --short has been deprecated, and will be removed in the future. The --short output will become the default.
Client Version: v1.25.3
Kustomize Version: v4.5.7
```

3. Instalar Minikube

1. Descargue el binario de Minikube ejecutando en PowerShell el siguiente comando:

```
New-Item -Path 'C:\minikube' -ItemType Directory
Invoke-WebRequest -Uri "https://github.com/kubernetes/minikube/releases/latest/download/minikube-windows-amd64.exe" -OutFile "C:\minikube\minikube.exe"
```

2. Agregue Minikube al PATH:
 - Al igual que con kubectl, agregue C:\minikube al PATH.
3. Verifique la instalación:

minikube version

4. Iniciar Minikube

1. Inicie Minikube especificando el controlador, por ejemplo con hyperv:

minikube start --driver=hyperv

Nota: Para Docker, usa --driver=docker.

2. Verifique que Minikube está en ejecución:

minikube status

```
PS C:\Users\Diego\Downloads\restoct\restoct> minikube status
* minikube 1.34.0 is available! Download it: https://github.com/kubernetes/minikube/releases/latest/download/minikube-windows-amd64.exe
* To disable this notice, run: 'minikube config set WantUpdateNotification false'

minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

Configura el contexto de kubectl:

- Cuando Minikube esté en ejecución, configure kubectl para usar el contexto de Minikube:

kubectl config use-context minikube

Para ver el contexto actual configurado en kubectl, puede utilizar los siguientes comandos en su terminal de PowerShell o cualquier otra terminal donde tenga configurado kubectl:

1. Ver el Contexto Actual:

kubectl config current-context

Este comando te muestra el nombre del contexto actualmente en uso.

```
PS C:\Users\Diego\Downloads\restoct\restoct> kubectl config current-context
minikube
```

2. Listar Todos los Contextos Disponibles:

kubectl config get-contexts

Este comando muestra una lista de todos los contextos configurados en tu archivo kubeconfig. La columna con * indica el contexto activo.

Curso: Arquitectura de Software

Laboratorio 3.

Universidad de Antioquia

```
PS C:\Users\Diego\Downloads\restoct\restoct> kubectl config get-contexts
CURRENT  NAME                                     CLUSTER                                     AUTHINFO                                     NAMESPACE
* gke_spring-boot-gke-353921_us-west1-a_spring-boot-cluster gke_spring-boot-gke-353921_us-west1-a_spring-boot-cluster gke_spring-boot-gke-353921_us-west1-a_spring-boot-cluster
gke_spring-boot-gke-353921_us-west1_ms-kb-cluster gke_spring-boot-gke-353921_us-west1_ms-kb-cluster gke_spring-boot-gke-353921_us-west1_ms-kb-cluster
minikube minikube minikube default
```

3. Ver Detalles del Contexto Actual:

kubectl config view --minify

Este comando muestra solo la configuración del contexto actualmente activo, incluyendo el clúster, el usuario y la información de namespace.

```
PS C:\Users\Diego\Downloads\restoct\restoct> kubectl config view --minify
apiVersion: v1
clusters:
- cluster:
    certificate-authority: C:\Users\Diego\.minikube\ca.crt
    extensions:
    - extension:
        last-update: Tue, 29 Oct 2024 11:24:02 -05
        provider: minikube.sigs.k8s.io
        version: v1.28.0
        name: cluster_info
        server: https://127.0.0.1:64575
      name: minikube
  contexts:
  - context:
      cluster: minikube
      extensions:
      - extension:
          last-update: Tue, 29 Oct 2024 11:24:02 -05
          provider: minikube.sigs.k8s.io
          version: v1.28.0
          name: context_info
          namespace: default
          user: minikube
        name: minikube
    current-context: minikube
  kind: Config
  preferences: {}
  users:
  - name: minikube
    user:
      client-certificate: C:\Users\Diego\.minikube\profiles\minikube\client.crt
      client-key: C:\Users\Diego\.minikube\profiles\minikube\client.key
```

4. Cambiar el Contexto Actual:

Si necesita cambiar a otro contexto de la lista, puede usar el siguiente comando:

kubectl config use-context <nombre-del-contexto>

Reemplace <nombre-del-contexto> con el nombre del contexto al que desea cambiar, según lo mostrado por `kubectl config get-contexts`.

5. Verificar la instalación de kubectl y Minikube

Use kubectl para listar los nodos en el clúster de Minikube:

kubectl get nodes

```
PS C:\Users\Diego\Downloads\restoct\restoct> kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
minikube    Ready     control-plane 62d   v1.25.3
```

Minikube, diseñado para ofrecer un entorno local simple y liviano, utiliza un solo nodo por defecto, lo cual facilita su configuración y uso en equipos personales para desarrollo y pruebas. Al funcionar con un solo nodo, Minikube optimiza el uso de recursos y reduce la complejidad. Sin embargo, en entornos de producción, es común utilizar múltiples nodos para distribuir las aplicaciones y garantizar tanto la alta disponibilidad como la escalabilidad del clúster. El nodo predeterminado en Minikube se denomina simplemente “minikube”.

Para ver el estado e información general del cluster puede dar el siguiente comando

kubectl cluster-info

```
PS C:\Users\Diego\Downloads\restoct\restoct> kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:64575
CoreDNS is running at https://127.0.0.1:64575/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
```

Curso: Arquitectura de Software

Laboratorio 3.

Universidad de Antioquia

Para configurar el entorno de Docker para que apunte al clúster de Minikube debe usar el siguiente comando:

minikube -p minikube docker-env | Invoke-Expression

o puede probar :

& minikube -p minikube docker-env | Invoke-Expression

Para confirmar que las variables de entorno están configuradas correctamente, ejecute:

docker ps

```
PS C:\Users\Diego\Downloads\restoct\restoct> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
aec86d354a64   gcr.io/k8s-minikube/kicbase:v0.0.36 "/usr/local/bin/entr..." 2 months ago   Up 34 hours   127.0.0.1:64571->22/tcp, 127.0.0.1:64572->2376/tcp, 127.0.0.1:64574->5000/tcp,
```

5.1 Construir el Proyecto Spring Boot

Primero, nos aseguramos de que el proyecto de Spring Boot esté construido correctamente. Navegue al directorio del proyecto y ejecute el siguiente comando desde cualquier consola:

.mvnw clean package

Este comando limpia el directorio de compilación y compila el proyecto, creando un archivo JAR que contiene la aplicación.

Después de ejecutar exitosamente el comando diríjase al directorio target de su proyecto y copie el archivo .jar generado y péguelo en la raíz de su proyecto.

Nota: Para evitar errores al momento de construir el JAR, elimine la clase Test de java.

5.2 Construir la Imagen de Docker

Después de construir el proyecto, el siguiente paso es crear la imagen de Docker de la aplicación. Esto se puede hacer con el siguiente comando:

docker build -t restoct-app:latest .

Este comando crea una imagen llamada restoct-app con la etiqueta latest, utilizando el Dockerfile presente en el directorio actual.

Verifique que se haya construido correctamente la imagen usando el comando **docker images ls**

```
PS C:\Users\Diego\Downloads\restoct\restoct> docker image ls
REPOSITORY      TAG       IMAGE ID       CREATED        SIZE
restoct-app     latest   081b71a7c404   24 hours ago   271MB
```

5.3 Crear Recursos en Kubernetes

Una vez que se ha construido la imagen de Docker, puedes desplegar los recursos en Kubernetes a través de los contenedores utilizando los archivos YAML creados anteriormente. Ejecute los siguientes comandos:

```
kubectl apply -f mysql-configmap.yml
kubectl apply -f mysql-deployment.yml
kubectl apply -f restoct-deployment.yml
```

Cada uno de estos comandos crea los recursos definidos en los archivos YAML, preparando el entorno para ejecutar la aplicación y la base de datos.

5.4 Verificar el Estado de los Pods

Para verificar que los pods se han creado y están en ejecución, utiliza el siguiente comando:

kubectl get pods

```
PS C:\Users\Diego\Downloads\restoct\restoct> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
mysql-9cf77964-qxg8w                1/1     Running   0           38h
restoct-app-64c87fb767-5gcrf        1/1     Running   0           38h
restoct-app-64c87fb767-ww8zh        1/1     Running   0           38h
```

Esto mostrará una lista de los pods en ejecución y su estado. Asegúrate de que todos los pods estén en estado Running.

Si quiere filtrar solo los pods relacionados a su aplicación puede usar el siguiente comando:

Curso: Arquitectura de Software
Laboratorio 3.
Universidad de Antioquia

kubectl get pods -l app=restoct

```
PS C:\Users\Diego\Downloads\restoct\restoct> kubectl get pods -l app=restoct
NAME                                READY   STATUS    RESTARTS   AGE
restoct-app-64c87fb767-5gcrf        1/1     Running   0           41h
restoct-app-64c87fb767-ww8zh        1/1     Running   0           41h
```

Otros comandos importantes a tener en cuenta:

Ver información detallada de los nodos

kubectl get nodes -o wide

```
PS C:\Users\Diego\Downloads\restoct\restoct> kubectl get nodes -o wide
NAME        STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION        CONTAINER-RUNTIME
minikube    Ready     control-plane 66d   v1.25.3   192.168.49.2   <none>         Ubuntu 20.04.5 LTS   5.10.16.3-microsoft-standard-WSL2   docker://20.10.20
```

Ver información detallada de todos los namespaces asociados a los pods.

Kubectl get pods --all-namespaces -o wide

```
PS C:\Users\Diego\Downloads\restoct\restoct> kubectl get pods --all-namespaces -o wide
NAMESPACE   NAME                                READY   STATUS    RESTARTS   AGE   IP             NODE        NOMINATED NODE   READINESS GATES
default     mysql-9cf77964-qxg8v               1/1     Running   0           4d22h  172.17.0.8     minikube    <none>            <none>
default     restoct-app-64c87fb767-5gcrf        1/1     Running   0           4d22h  172.17.0.5     minikube    <none>            <none>
default     restoct-app-64c87fb767-ww8zh        1/1     Running   0           4d22h  172.17.0.6     minikube    <none>            <none>
kube-system coredns-56cd847f94-fkqbh           1/1     Running   2 (6d22h ago)  66d   172.17.0.2     minikube    <none>            <none>
kube-system etcd-minikube                       1/1     Running   2 (6d22h ago)  66d   192.168.49.2   minikube    <none>            <none>
kube-system kube-apiserver-minikube      1/1     Running   2 (6d22h ago)  66d   192.168.49.2   minikube    <none>            <none>
kube-system kube-controller-manager-minikube  1/1     Running   2 (6d22h ago)  66d   192.168.49.2   minikube    <none>            <none>
kube-system kube-proxy-4h5bj             1/1     Running   2 (6d22h ago)  66d   192.168.49.2   minikube    <none>            <none>
kube-system kube-scheduler-minikube        1/1     Running   2 (6d22h ago)  66d   192.168.49.2   minikube    <none>            <none>
kube-system storage-provisioner           1/1     Running   37 (82m ago)   66d   192.168.49.2   minikube    <none>            <none>
kubernetes-dashboard dashboard-metrics-scraper-b74747df5-r944f  1/1     Running   0           4d22h  172.17.0.7     minikube    <none>            <none>
kubernetes-dashboard kubernetes-dashboard-57bbdc5f89-8kg7j    1/1     Running   0           4d22h  172.17.0.9     minikube    <none>            <none>
PS C:\Users\Diego\Downloads\restoct\restoct>
```

Ver el log detallado del contenedor del pod:

kubectl logs nombre_pod

eliminar el config map de mysql

kubectl delete configmap mysql-config

borrar un pod en específico

kubectl delete pod nombre_pod

borrar varios pods

kubectl delete pods -l app=nombre_pod_inicial (Ejemplo kubectl delete pods -l app=restoct)

eliminar la aplicación desplegada en los pods

kubectl delete deployment restoct-app

ver información en detalle del pod

kubectl describe pod nombre_pod

Obtener información detallada sobre un nodo específico

kubectl describe node nombre_nodo

ver información detallada del configmap

kubectl get configmap mysql-config -o yaml

agregar propiedades de MySQL sobre el configmap

kubectl create configmap mysql-config --from-literal=MYSQL_USER=root --from-literal=MYSQL_PASSWORD=su_contraseña

Confirma que el ConfigMap contiene todas las claves necesarias (MYSQL_DATABASE, MYSQL_ROOT_PASSWORD, MYSQL_USER, MYSQL_PASSWORD).

Inspecciona el ConfigMap: **kubectl get configmap mysql-config -n default -o yaml**

Revisar los logs del nodo de MySQL

kubectl logs mysql-<ID> -n default

Editar el configmap de MySQL

kubectl edit configmap mysql-config -n default

Curso: Arquitectura de Software

Laboratorio 3.

Universidad de Antioquia

ver la dirección IP de Minikube
minikube ip

Listar todos los namespaces en el clúster

kubectl get ns

```
PS C:\Users\Diego\Downloads\restoct\restoct> kubectl get ns
NAME                STATUS   AGE
default             Active   66d
kube-node-lease     Active   66d
kube-public         Active   66d
kube-system         Active   66d
kubernetes-dashboard Active   66d
```

Este comando mostrará una lista de los namespaces disponibles en su clúster de Kubernetes, indicando su estado y otra información relevante. Así obtendrá una visión general de los espacios lógicos creados en el clúster, los cuales ayudan a organizar y aislar aplicaciones y recursos. Es importante notar que Minikube incluye algunos namespaces predeterminados para facilitar el trabajo.

En Kubernetes, el namespace "default" es el espacio de nombres predeterminado donde se despliegan aplicaciones y recursos cuando no se especifica otro namespace explícitamente. Es el lugar al que se asignan por defecto los recursos creados sin un namespace definido.

Obtener una lista de los pods que están corriendo en un namespace específico en Kubernetes

kubectl get pods -n nombre-del-namespace

```
PS C:\Users\Diego\Downloads\restoct\restoct> kubectl get pods -n default
NAME                                READY   STATUS    RESTARTS   AGE
mysql-9cf77964-qxg8w               1/1     Running   0           4d23h
restoct-app-64c87fb767-5gcrf       1/1     Running   0           4d22h
restoct-app-64c87fb767-ww8zh       1/1     Running   0           4d22h
```

Otros comandos para revisar:

kubectl get svc → retorna la información general de los servicios asociados de kubernetes.

5.5 Acceder a la Aplicación REST

Para acceder a la aplicación REST, necesitas obtener la dirección IP del servicio que has creado. Utiliza el siguiente comando para obtener la información del servicio:

kubectl get service restoct-service

Este comando proporciona una dirección IP externa a la que puede acceder desde su navegador o herramientas como Postman para probar los endpoints de la aplicación.

Si está usando Minikube, obtenga la URL para acceder al servicio Spring Boot ejecutando:

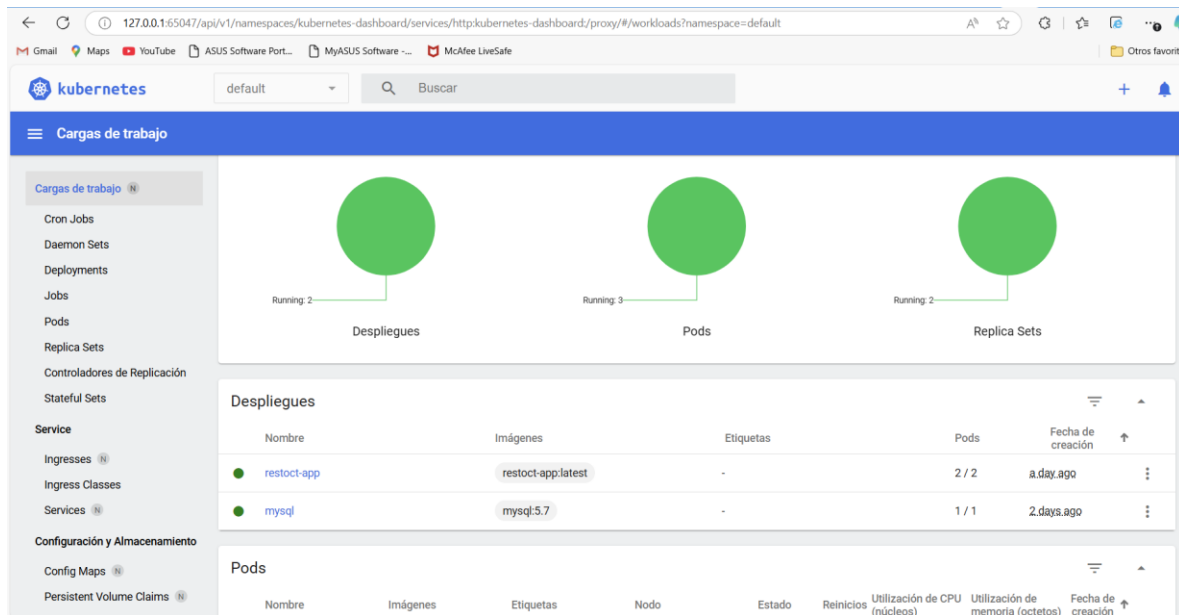
minikube service restoct-service

```
PS C:\Users\Diego\Downloads\restoct\restoct> minikube service restoct-service
|-----|-----|-----|-----|
| NAMESPACE | NAME       | TARGET PORT | URL                               |
|-----|-----|-----|-----|
| default   | restoct-service | 8089        | http://192.168.49.2:31616        |
|-----|-----|-----|-----|
* Starting tunnel for service restoct-service.
|-----|-----|-----|-----|
| NAMESPACE | NAME       | TARGET PORT | URL                               |
|-----|-----|-----|-----|
| default   | restoct-service |            | http://127.0.0.1:55592           |
|-----|-----|-----|-----|
* Opening service default/restoct-service in default browser...
! Porque estás usando controlador Docker en windows, la terminal debe abrirse para ejecutarlo.
```

Esto abrirá la URL en el navegador o le proporcionará la dirección IP y puerto para acceder a la aplicación. Su aplicación Spring Boot debería estar disponible en esa URL.

Para abrir el dashboard de minikube puede ejecutar el comando **minikube dashboard**

Curso: Arquitectura de Software
Laboratorio 3.
Universidad de Antioquia



Nota: Si al momento de ejecutar el comando sale el siguiente error:

PS C:\Users\Diego\Downloads\restoct\restoct> **minikube dashboard** * Habilitando dashboard - Using image docker.io/kubernetesui/dashboard:v2.7.0 - Using image docker.io/kubernetesui/metrics-scraper:v1.0.8 X Saliendo por un error SVC_DASHBOARD_ROLE_REF: run callbacks: running callbacks: [sudo KUBECONFIG=/var/lib/minikube/kubeconfig

stderr: The ClusterRoleBinding "kubernetes-dashboard" is invalid: roleRef: Invalid value: rbac.RoleRef{APIGroup:"rbac.authorization.k8s.io", Kind:"ClusterRole", Name:"cluster-admin"}: cannot change roleRef] * Suggestion: Run: 'kubectl delete clusterrolebinding kubernetes-dashboard' * Related issue: <https://github.com/kubernetes/minikube/issues/7256>

Indica que Minikube tiene problemas al aplicar ciertas configuraciones para el dashboard de Kubernetes, específicamente con la variable ClusterRoleBinding llamada kubernetes-dashboard. Para resolver esto, sigue estos pasos:

1. **Elimina la ClusterRoleBinding existente.** Ejecute el siguiente comando para eliminar la ClusterRoleBinding conflictiva:

kubectl delete clusterrolebinding kubernetes-dashboard

Para ver todos los recursos del cluster puede usar el comando:

kubectl get all

```
PS C:\Users\Diego\Downloads\restoct\restoct> kubectl get all
NAME                                     READY   STATUS    RESTARTS   AGE
pod/mysql-9cf77964-qxg8w                1/1     Running   0           44h
pod/restoct-app-64c87fb767-5gcrf        1/1     Running   0           43h
pod/restoct-app-64c87fb767-ww8zh        1/1     Running   0           43h

NAME                                     TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
service/kubernetes                      ClusterIP      10.96.0.1    <none>         443/TCP          63d
service/mysql                           ClusterIP      10.105.29.29 <none>         3306/TCP          2d6h
service/restoct-service                  LoadBalancer  10.101.45.85 <pending>     8089:31616/TCP   2d6h

NAME                                     READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/mysql                   1/1     1             1           2d6h
deployment.apps/restoct-app              2/2     2             2           43h

NAME                                     DESIRED   CURRENT   READY   AGE
replicaset.apps/mysql-9cf77964          1         1         1       2d6h
replicaset.apps/restoct-app-64c87fb767  2         2         2       43h
```

Para verificar los objetos desplegados puede usar el comando:

Curso: Arquitectura de Software
Laboratorio 3.
Universidad de Antioquia

kubectl get deployments

```
PS C:\Users\Diego\Downloads\restoct\restoct> kubectl get deployments
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
mysql         1/1     1            1           5d9h
restoct-app   2/2     2            2           4d23h
```

Acceso a la Base de Datos

Para acceder a la base de datos MySQL dentro del pod en Kubernetes, puede seguir estos pasos:

1. Ejecuta un shell en el pod de MySQL. Use kubectl exec para abrir una terminal en el pod que está ejecutando MySQL.

kubectl exec -it nombre_pod_mysql -- /bin/bash

2. Acceda a MySQL desde el shell. Una vez dentro del pod, puede utilizar el cliente de MySQL. Ejecute el siguiente comando reemplazando root y su_password por el nombre de usuario y contraseña correctos:

mysql -u root -p

Después, ingrese la contraseña cuando se la pida.

3. Verifica la base de datos. Una vez conectado al cliente de MySQL, puedes listar las bases de datos, tablas y realizar cualquier operación SQL que necesite.

SHOW DATABASES;

4. Salir del cliente MySQL y del shell del pod. Use exit para salir de MySQL y nuevamente exit para salir del pod.

Curso: Arquitectura de Software
Laboratorio 3.
Universidad de Antioquia

Administrador: Windows PowerShell

```
PS C:\Users\Diego\Downloads\restoct\restoct> kubectl exec -it mysql-9cf77964-qxg8w -- /bin/bash
bash-4.2# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 22
Server version: 5.7.44 MySQL Community Server (GPL)

Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| restoct |
| sys |
+-----+
5 rows in set (0.04 sec)

mysql> use restoct;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> show tables;
+-----+
| Tables_in_restoct |
+-----+
| flight |
| hibernate_sequence |
+-----+
2 rows in set (0.00 sec)

mysql> select * from flight;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| idflight | capacidad | cumplido | destino | nombreavion | numerovuelo | origen | planvuelo | rating |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | 180 | 0 | MDE | A320 | AV333 | BOG | 332332 | 3 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> exit
Bye
bash-4.2# exit
exit
```

7. Conclusiones

Al finalizar esta guía de laboratorio, los estudiantes habrán adquirido las habilidades necesarias para:

1. **Desarrollar Aplicaciones REST:** Ser capaces de crear aplicaciones utilizando el framework Spring Boot e implementar principios de HATEOAS.
2. **Contenerizar Aplicaciones:** Comprender cómo utilizar Docker para contenerizar aplicaciones y bases de datos, facilitando la portabilidad.
3. **Desplegar en Kubernetes:** Familiarizarse con la creación y gestión de despliegues y servicios en un entorno de Kubernetes.
4. **Realizar Pruebas Funcionales:** Ejecutar pruebas para validar que la aplicación REST cumple con los requisitos y que los servicios funcionan correctamente.

Curso: Arquitectura de Software
Laboratorio 3.
Universidad de Antioquia

8. Recursos Adicionales

Para profundizar más en los temas tratados en esta guía, se recomienda revisar los siguientes recursos:

1. **Documentación de Spring Boot:** [Spring Boot Documentation](#)
2. **Introducción a Docker:** Docker Documentation
3. **Kubernetes para Principiantes:** Kubernetes Documentation
4. **HATEOAS y REST:** HATEOAS
5. **MySQL Documentation:** [MySQL Documentation](#)

9. Ejercicios Prácticos **FECHA DE ENTREGA 22 de Mayo de 2026**

9.1 ArgoCD. Instale y configure ArgoCD para desplegar PODs usando la estrategia GitOps.

9.2 Escalabilidad

Experimente con el escalado horizontal de la aplicación REST aumentando el número de réplicas en el Deployment. Observe cómo Kubernetes gestiona el tráfico y la carga. Prueba con herramientas como JMETTER u otras que considere para hacer la prueba de carga y de escalabilidad.

9.3 Monitoreo (Ejercicio Opcional)

Configura herramientas de monitoreo como Prometheus y Grafana para supervisar el estado y rendimiento de la aplicación y la base de datos.